

Proyecto ADC + DMA

Lecturas Analógicas y Visualización Gráfica

STM32F4 Discovery +

SDBoard

Altuna, Iñaki

Pais, Tomas

Pose, Mateo

| DESCRIPCIÓN DEL SISTEMA



Hardware: STM32F407G-DISC1 con expansión SDBoard.



Variables: Lectura de un sensor de temperatura y un potenciómetro.



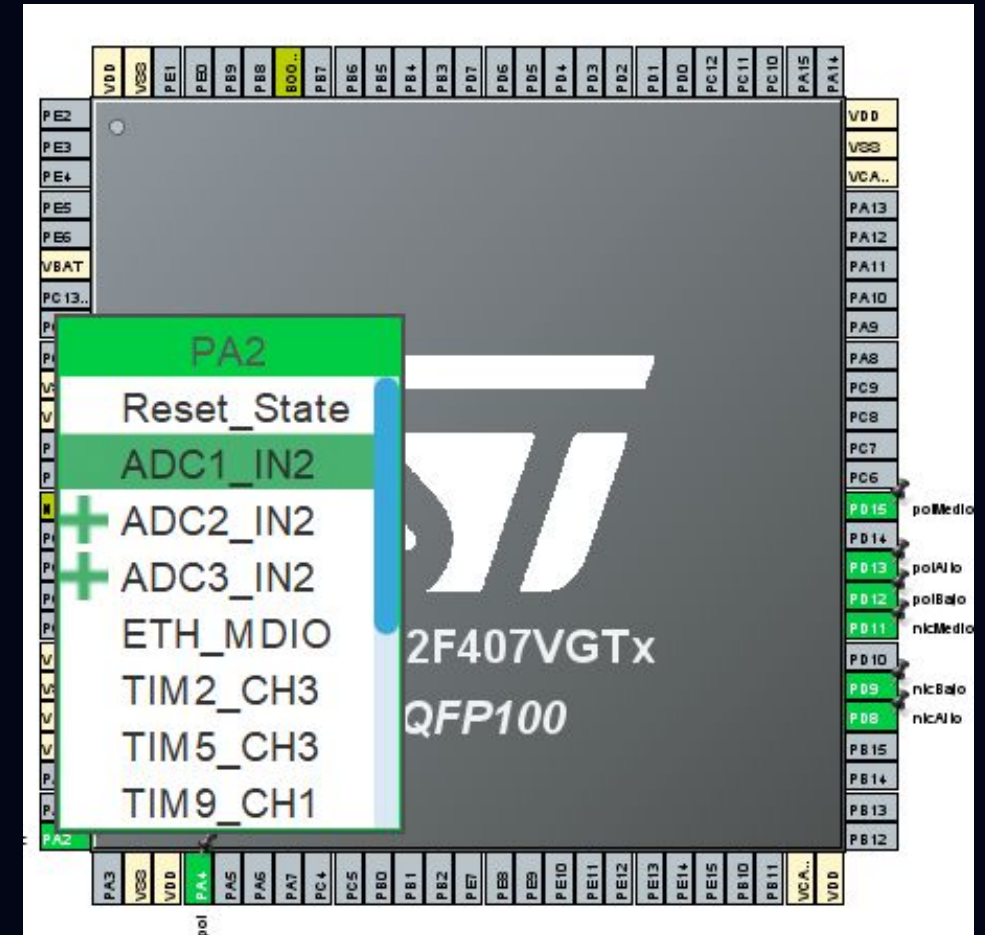
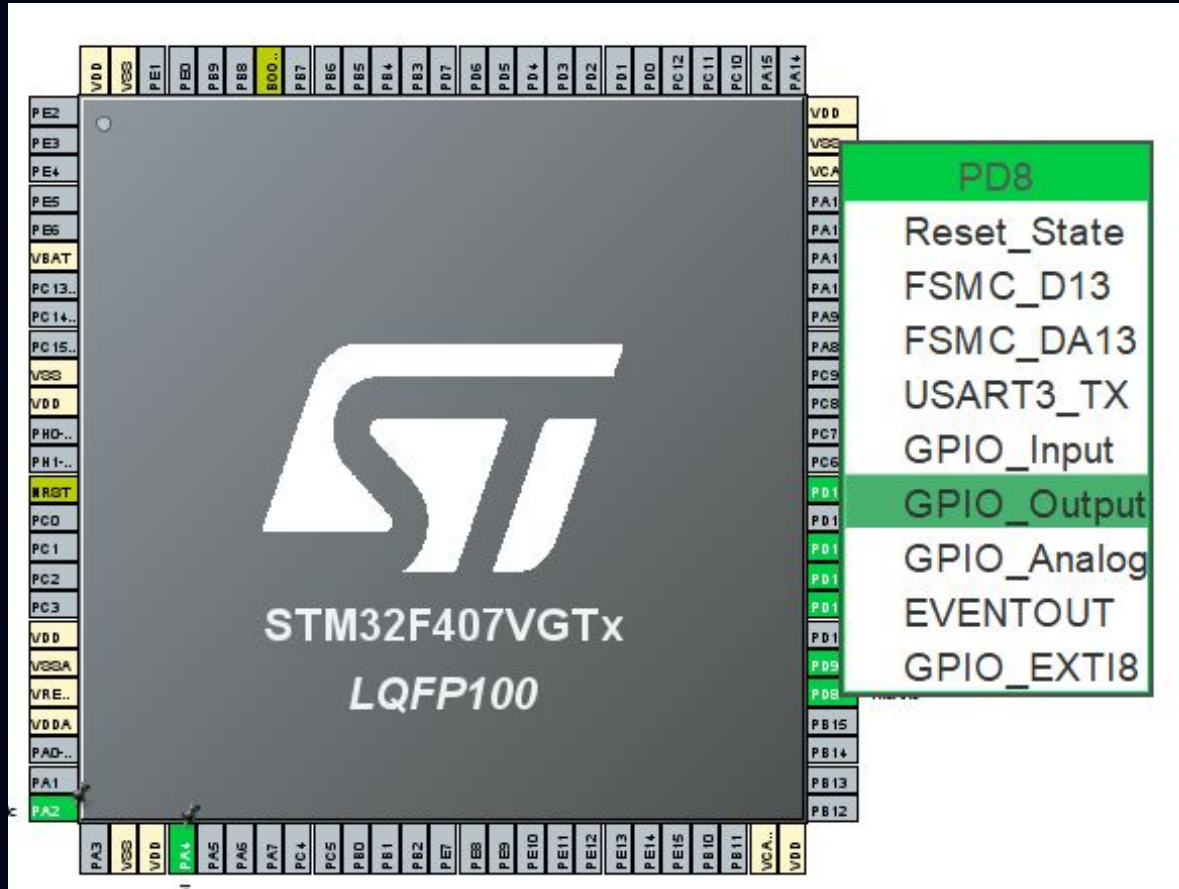
Visualización: Clasificación en rangos Bajo, Medio y Alto mediante columnas de LEDs.



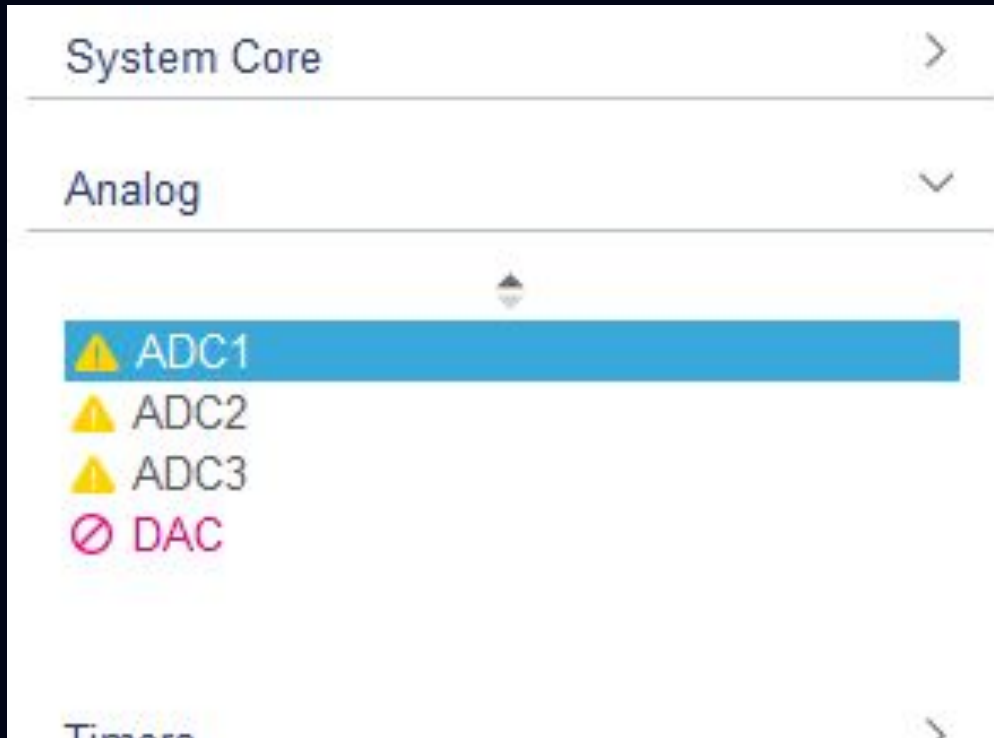
| DESAFÍO

- Utilizar el ADC en modo continuo con DMA
- Utilizar el potenciómetro para regular la velocidad de parpadeo de los LEDs
- Utilizar la NTC para regular la cantidad de LEDs que parpadean
- Ajustar la variación del potenciómetro a un rango de tiempos de parpadeo entre 0 y 5 segundos
- Ajustar correctamente la cantidad de LEDs encendidos

CONFIGURACIÓN DE PINES



| CONFIGURACIÓN DEL ADC1



▼ ADC_Settings	
Clock Prescaler	PCLK2 divided by 8
Resolution	12 bits (15 ADC Clock cycles)
Data Alignment	Right alignment
Scan Conversion Mode	Enabled
Continuous Conversion Mode	Enabled
Discontinuous Conversion M...	Disabled
DMA Continuous Requests	Enabled
End Of Conversion Selection	EOC flag at the end of all conversions

| DMA: GESTIÓN INTELIGENTE

El **DMA2 Stream 0** actúa como un puente directo entre el periférico y la memoria RAM:

Modo	Circular (Sin fin)
Dirección	Periférico a Memoria
Ancho de datos	Half Word (16 bits)
Incremento	Habilitado en Memoria

Ventaja Crítica

La CPU queda 100% liberada de la tarea de mover datos. Solo se despierta cuando el buffer está listo para ser procesado.

| PROMEDIADO DE MUESTRAS

DMA recopila dos conjuntos de 16 muestras a través de sus dos canales. Uno para el ADC y otro para el NTC.

Se genera una interrupción cada vez que el DMA junta 16 muestras de cada canal, las cuales se promedian y se mandan directamente a la memoria.

```
// Rutina de atención a la interrupción de buffer a mitad
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc) {
// Promediamos la primera mitad del buffer (el primer bloque de tamaño mínimo)
promediar(&dma_buffer[0]);
}
// Rutina de atención a la interrupción de buffer a tope
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
// Promediamos la segunda mitad del buffer (el segundo bloque de tamaño mínimo)
promediar(&dma_buffer[BUFFER_MINIMO]);
}
```

```
void promediar(uint16_t *buffer) {
    uint32_t ntc_sum = 0, pot_sum = 0;
    for (int i = 0; i < MUESTRAS; ++i) {
        ntc_sum += buffer[0];
        pot_sum += buffer[1];
        buffer += 2;
    }
    ntc_avg = ntc_sum / MUESTRAS;
    pot_avg = pot_sum / MUESTRAS;
}
```

| CONFIGURACIÓN LEDS Y VELOCIDAD

Selección número de leds

```
#define NTC_FRIO 3700  
#define NTC_CALIENTE 3400
```

```
uint16_t ntc_saturada = ntc_avg;  
  
if (ntc_saturada > NTC_FRIO) {  
    ntc_saturada = NTC_FRIO;  
}  
else if (ntc_saturada < NTC_CALIENTE) {  
    ntc_saturada = NTC_CALIENTE;  
}  
  
uint8_t cantidad_leds = 6 - (((ntc_saturada - NTC_CALIENTE) * 6) / (NTC_FRIO - NTC_CALIENTE));
```

Selección velocidad de parpadeo

```
#define POT_MAX_LSB 4095  
#define TIEMPO_MAX_MS 5000
```

```
uint32_t tiempo_parpadeo = (pot_avg * TIEMPO_MAX_MS) / POT_MAX_LSB;
```


| CONTROL DE LEDS

El bit set reset register (BSRR) está compuesto por 32 bits del 0 al 15 controlan el encendido (set)y del 16 al 31 el apagado (reset)

```
static uint32_t ultimo_cambio = 0;
static uint8_t estado_leds = 0;

if ((HAL_GetTick() - ultimo_cambio) >= tiempo_parpadeo) {
    ultimo_cambio = HAL_GetTick();
    estado_leds = !estado_leds;

    if (tiempo_parpadeo < 50) {
        estado_leds = 1;
    }

    actualizarLEDs(cantidad_leds, estado_leds);
}
```

```
void actualizarLEDs(uint8_t cantidad, uint8_t estado) {
    apagarTodos();

    if (estado == 1 && cantidad > 0) {
        uint32_t mascara = 0;
        for(int i = 0; i < cantidad; i++) {
            if(i < 6) {
                mascara |= pines_leds[i];
            }
        }
        leds_OUT_Port->BSRR = mascara;
    }
}
```

```
void apagarTodos(void){
    leds_OUT_Port->BSRR = MASCARA_APAGAR_LEDS;
}
```

```
#define MASCARA_APAGAR_LEDS ((ntcAlto_Pin|ntcBajo_Pin|ntcMedio_Pin|potBajo_Pin|potAlto_Pin|potMedio_Pin) << 16)
```

| INTEGRACIÓN ADC + DMA

Resumen de Flujo

- 1 ADC captura datos en segundo plano.
- 2 DMA traslada a RAM automáticamente.
- 3 CPU procesa promedios solo por interrupción.
- 4 Main Loop decide visualización y tiempos.



¿Preguntas?

Altuna, Pais , Pose | Sistemas Digitales
2026
